

Lecture 13: Graph Modeling and Ingestion

Topics

- What is a graph? Vertices and edges
- Edge list representation: simple and flexible
- Adjacency list with dictionaries: fast neighbor lookup
- Adjacency matrix: when dense graphs make sense
- Loading graphs from CSV and JSON data
- Building real-world graphs: artist collaboration, track similarity

Goals

- Understand graph terminology and when to use graphs
- Represent graphs as edge lists, adjacency lists, and matrices
- Load and build graphs from real data
- Choose the right representation for each use case
- Model Spotify data as artist collaboration networks

Roadmap

First half (~35 min)

- What is a graph? Real-world examples
- Graph terminology: vertices, edges, directed vs undirected
- Edge list representation: tuples and lists
- Adjacency list with dictionaries: neighbor lookup
- Building graphs from edge lists
- In-class exercise 1 (commit required)

Break (3 min)

Second half (~35 min)

- Adjacency matrix: 2D array representation
- Representation comparison: space and time tradeoffs
- Loading graphs from CSV files
- Loading graphs from JSON data
- Building Spotify artist collaboration network
- In-class exercise 2 (commit required)
- Complexity table and wrap-up

Setup: load Spotify data

```
In [ ]: import csv
import json
from collections import defaultdict
from typing import List, Dict, Set, Tuple

# Load tracks
tracks = {}
with open('data/tracks.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        # Normalize track_id (strip whitespace, lowercase)
        track_id = row['track_id'].strip().lower()
        tracks[track_id] = row

# Load artists
artists = {}
with open('data/artists.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        artists[row['artist_id']] = row

# Load plays
plays = []
with open('data/plays.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
```

```
        plays.append(row)

# Add artist names to tracks for easier access
for track_id, track in tracks.items():
    artist_id = track['artist_id']
    if artist_id in artists:
        track['artist'] = artists[artist_id]['artist_name']
    else:
        track['artist'] = artist_id # Fallback to ID

print(f"Loaded {len(tracks)} tracks, {len(artists)} artists, and {len(p"
```

Part 1: What is a Graph?

A graph is a collection of nodes (vertices) connected by links (edges).

Real-world examples:

- **Social networks:** People (nodes) are friends (edges)
- **Road networks:** Cities (nodes) connected by roads (edges)
- **Web pages:** Pages (nodes) linked by hyperlinks (edges)
- **Music:** Artists (nodes) who collaborated (edges)
- **Recommendations:** Tracks (nodes) that are similar (edges)

Why graphs?

- Model relationships between entities
- Find paths and connections
- Analyze network structure
- Make recommendations

Graph terminology

Vertex (Node): A single entity in the graph

- Example: "Taylor Swift", "Ed Sheeran"

Edge (Link): A connection between two vertices

- Example: ("Taylor Swift", "Ed Sheeran") — they collaborated

Degree: The number of edges connected to a vertex

- Example: If Taylor Swift has 5 collaborations, her degree is 5
- For directed graphs: **in-degree** (incoming edges) and **out-degree** (outgoing edges)

Directed vs Undirected:

- **Undirected:** Edge has no direction (friendship, collaboration)
 - (A, B) same as (B, A)
- **Directed:** Edge has direction (following, hyperlinks)
 - (A, B) means $A \rightarrow B$
 - (B, A) means $B \rightarrow A$ (different!)

Weighted vs Unweighted:

- **Unweighted:** All edges are equal
- **Weighted:** Edges have values (distance, strength, similarity)

Spotify as a graph: examples

Artist collaboration graph (undirected)

```
Vertices: Artists
Edges: Two artists collaborated on a track

"Taylor Swift" --- "Ed Sheeran"
  |                 |
"Bon Iver"         "Justin Bieber"
```

Track similarity graph (undirected, weighted)

```
Vertices: Tracks
Edges: Tracks are similar (weight = similarity score)

"Track A" --(0.8)-- "Track B"
  |                 |
(0.6)             (0.9)
  |                 |
"Track C" --(0.7)-- "Track D"
```

User playlist graph (directed)

```
Vertices: Users
Edges: User A follows user B's playlists

Alice → Bob → Carol
  ↑         |
  |         |
  |         |
```

Edge List Representation

Simplest representation: Just store all the edges.

Edge list: List of tuples (vertex1, vertex2) or (vertex1, vertex2, weight)

```
edges = [  
    ("Alice", "Bob"),  
    ("Bob", "Carol"),  
    ("Alice", "Carol"),  
]
```

Advantages:

- Simple to understand and implement
- Compact for sparse graphs (few edges)
- Easy to add/remove edges

Disadvantages:

- Finding neighbors is slow: $O(E)$ where E = number of edges
- Checking if edge exists is slow: $O(E)$

Edge list example: artist collaborations

```
In [ ]: def build_collaboration_edges(tracks: dict) -> List[Tuple[str, str]]:
        """Build edge list of artist collaborations."""
        edges = []

        for track_id, track in tracks.items():
            artist = track['artist']
            # Look for features (simplified: just check if artist field has
            if '&' in artist or 'feat.' in artist.lower():
                # Simplified: split by common separators
                artists = artist.replace('feat.', '&').replace('Feat.', '&')
                artists = [a.strip() for a in artists]

                # Create edges between all pairs (undirected)
                for i in range(len(artists)):
                    for j in range(i + 1, len(artists)):
                        edges.append((artists[i], artists[j]))

        return edges

# Build collaboration edges
collab_edges = build_collaboration_edges(tracks)
print(f"Found {len(collab_edges)} collaboration edges")
print("\nSample edges:")
for edge in collab_edges[:5]:
    print(f" {edge[0]} <-> {edge[1]}")
```

Operations on edge lists

Count vertices:

```
In [ ]: def count_vertices(edges: List[Tuple[str, str]]) -> int:
        """Count unique vertices in edge list."""
        vertices = set()
        for v1, v2 in edges:
            vertices.add(v1)
            vertices.add(v2)
        return len(vertices)

num_vertices = count_vertices(collab_edges)
print(f"Number of artists who collaborated: {num_vertices}")
```


Operations on edge lists

Count vertices:

```
In [ ]: def count_vertices(edges: List[Tuple[str, str]]) -> int:
        """Count unique vertices in edge list."""
        vertices = set()
        for v1, v2 in edges:
            vertices.add(v1)
            vertices.add(v2)
        return len(vertices)

num_vertices = count_vertices(collab_edges)
print(f"Number of artists who collaborated: {num_vertices}")
```

Find neighbors (slow!):

```
In [ ]: def find_neighbors_slow(edges: List[Tuple[str, str]], vertex: str) -> List[str]:
        """Find all neighbors of a vertex. O(E) time!"""
        neighbors = []
        for v1, v2 in edges:
            if v1 == vertex:
                neighbors.append(v2)
            elif v2 == vertex: # Undirected
                neighbors.append(v1)
        return neighbors
```

```
# Find who collaborated with first artist
if collab_edges:
    first_artist = collab_edges[0][0]
    neighbors = find_neighbors_slow(collab_edges, first_artist)
    print(f"\n{first_artist} collaborated with: {neighbors}")
```

Adjacency List with Dictionaries

Better representation for neighbor queries!

Adjacency list: Dictionary mapping each vertex to its list of neighbors.

```
graph = {  
    "Alice": ["Bob", "Carol"],  
    "Bob": ["Alice", "Carol"],  
    "Carol": ["Alice", "Bob"],  
}
```

Advantages:

- Fast neighbor lookup: $O(1)$ to find neighbors
- Efficient for sparse graphs (most graphs are sparse!)
- Easy to iterate over neighbors

Disadvantages:

- Uses more memory than edge list
- Checking if edge exists still $O(\text{degree})$ unless using sets

Building adjacency list from edge list

```
In [ ]: def build_adjacency_list(edges: List[Tuple[str, str]],
                                   directed: bool = False) -> Dict[str, List[str]]
        """Build adjacency list from edge list."""
        graph = defaultdict(list)

        for v1, v2 in edges:
            graph[v1].append(v2)
            if not directed:
                # Undirected: add edge in both directions
                graph[v2].append(v1)

        return dict(graph)

# Build adjacency list for collaborations
collab_graph = build_adjacency_list(collab_edges, directed=False)

print(f"Graph has {len(collab_graph)} vertices\n")
print("Sample neighbors:")
for i, (artist, neighbors) in enumerate(list(collab_graph.items())[:3]):
    print(f" {artist}: {neighbors}")
```

Operations on adjacency lists

Find neighbors (fast!):

```
In [ ]: def get_neighbors(graph: Dict[str, List[str]], vertex: str) -> List[str]:
        """Get neighbors of vertex. O(1) dictionary lookup!"""
        return graph.get(vertex, [])

        # Much faster than edge list!
        if collab_graph:
            first_artist = list(collab_graph.keys())[0]
            neighbors = get_neighbors(collab_graph, first_artist)
            print(f"{first_artist} has {len(neighbors)} collaborators")
```

Operations on adjacency lists

Find neighbors (fast!):

```
In [ ]: def get_neighbors(graph: Dict[str, List[str]], vertex: str) -> List[str]:
        """Get neighbors of vertex. O(1) dictionary lookup!"""
        return graph.get(vertex, [])

        # Much faster than edge list!
        if collab_graph:
            first_artist = list(collab_graph.keys())[0]
            neighbors = get_neighbors(collab_graph, first_artist)
            print(f"{first_artist} has {len(neighbors)} collaborators")
```

Compute degree (number of neighbors):

```
In [ ]: def compute_degrees(graph: Dict[str, List[str]]) -> Dict[str, int]:
        """Compute degree of each vertex."""
        return {vertex: len(neighbors) for vertex, neighbors in graph.items}

        degrees = compute_degrees(collab_graph)
        if degrees:
            # Find artist with most collaborations
            max_artist = max(degrees.items(), key=lambda x: x[1])
            print(f"\nMost collaborative artist: {max_artist[0]} "
                  f"({max_artist[1]} collaborations)")
```

Using sets for faster edge checks

Problem: Checking if edge exists in adjacency list is $O(\text{degree})$

Solution: Use sets instead of lists for neighbors

```
In [ ]: def build_adjacency_set(edges: List[Tuple[str, str]],
                                directed: bool = False) -> Dict[str, Set[str]]:
    """Build adjacency list with sets for  $O(1)$  edge checks."""
    graph = defaultdict(set)

    for v1, v2 in edges:
        graph[v1].add(v2)
        if not directed:
            graph[v2].add(v1)

    return dict(graph)

def has_edge(graph: Dict[str, Set[str]], v1: str, v2: str) -> bool:
    """Check if edge exists.  $O(1)$  with sets!"""
    return v2 in graph.get(v1, set())

# Build with sets
collab_set_graph = build_adjacency_set(collab_edges)

# Fast edge check
if len(collab_edges) >= 1:
```

```
v1, v2 = collab_edges[0]  
print(f"Edge ({v1}, {v2}) exists? {has_edge(collab_set_graph, v1, v2)}")
```

Exercise 1: Build track similarity graph

Task: Build a graph of similar tracks based on shared genre or artist.

Requirements:

1. Create edges between tracks by the same artist
2. Build adjacency list (use sets for neighbors)
3. Find the track with the most similar tracks (highest degree)
4. Print: track name, artist, and number of similar tracks

Hint: Group tracks by artist first, then create edges within each group.

Commit required: Commit your solution before the break.

```
In [ ]: # YOUR CODE HERE
        # Build track similarity graph and find most connected track

def build_track_similarity_graph(tracks):
    """Build graph where tracks by same artist are connected."""
    # TODO: implement
    pass

# Build graph and find most connected track
# track_graph = build_track_similarity_graph(tracks)
```

Break (3 minutes)

Commit your Exercise 1 solution now!

When we return:

- Adjacency matrix representation
- Loading graphs from CSV and JSON
- Representation tradeoffs

Part 2: Adjacency Matrix

2D array representation: `matrix[i][j] = 1` if edge (i, j) exists

```
# Graph: A-B, B-C, A-C
vertices = ['A', 'B', 'C']
matrix = [
    [0, 1, 1], # A connects to B, C
    [1, 0, 1], # B connects to A, C
    [1, 1, 0], # C connects to A, B
]
```

For weighted graphs: Store weight instead of 1

Advantages:

- $O(1)$ edge existence check: `matrix[i][j]`
- $O(1)$ edge weight lookup
- Simple for dense graphs

Disadvantages:

- $O(V^2)$ space even for sparse graphs!
- Finding neighbors still $O(V)$ (scan entire row)
- Not practical for large sparse graphs

Building adjacency matrix

```
In [ ]: def build_adjacency_matrix(edges: List[Tuple[str, str]],
                                     directed: bool = False) -> Tuple[List[List[int]],
                                     Dict[str, int]]:
    """Build adjacency matrix from edge list."""
    # Get unique vertices and map to indices
    vertices = set()
    for v1, v2 in edges:
        vertices.add(v1)
        vertices.add(v2)

    vertex_to_idx = {v: i for i, v in enumerate(sorted(vertices))}
    n = len(vertices)

    # Initialize matrix
    matrix = [[0] * n for _ in range(n)]

    # Fill matrix
    for v1, v2 in edges:
        i, j = vertex_to_idx[v1], vertex_to_idx[v2]
        matrix[i][j] = 1
        if not directed:
            matrix[j][i] = 1

    return matrix, vertex_to_idx

# Build matrix (use small sample to avoid huge matrix)
sample_edges = collab_edges[:20] # Small sample
```

```
matrix, vertex_map = build_adjacency_matrix(sample_edges)

print(f"Matrix size: {len(matrix)} × {len(matrix)} = {len(matrix)**2} cells")
print(f"Actual edges: {len(sample_edges)}")
print(f"Space efficiency: {len(sample_edges) / (len(matrix)**2):.2%}")
```

When to use each representation

Representation	Space	Edge Check	Get Neighbors	Best For
Edge List	$O(E)$	$O(E)$	$O(E)$	Sparse, few queries
Adjacency List (list)	$O(V + E)$	$O(\text{degree})$	$O(1)$	Most graphs!
Adjacency List (set)	$O(V + E)$	$O(1)$	$O(1)$	Need fast edge checks
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(V)$	Dense graphs only

Sparse graph: Few edges compared to V^2 (most real-world graphs)

- Social networks: average person has <1000 friends, but millions of people
- Web: pages link to handful of others, not all pages
- **Use adjacency list**

Dense graph: Many edges (close to V^2)

- Complete graph: every vertex connected to every other
- Small fully-connected clusters
- **Adjacency matrix OK**

Default choice: Adjacency list with lists (or sets if need fast edge checks)

Loading Graphs from Files

Real-world graphs come from data files:

- CSV files with edge lists
- JSON files with graph structures
- Database exports
- API responses

Loading from CSV: edge list format

```
In [ ]: def load_graph_from_csv(filename: str,
                                source_col: str,
                                target_col: str,
                                directed: bool = False) -> Dict[str, Set[str]]:
    """Load graph from CSV file with edge list."""
    graph = defaultdict(set)

    with open(filename, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            source = row[source_col]
            target = row[target_col]

            graph[source].add(target)
            if not directed:
                graph[target].add(source)

    return dict(graph)

# Example: Build graph from plays data
# Create a "user-track" bipartite graph
# (For demo, we'll build it in memory since we already have plays)

def build_user_track_graph(plays: list) -> Dict[str, Set[str]]:
    """Build bipartite graph: users connected to tracks they played.
```

Bipartite graph: A graph where vertices can be divided into two disjoint sets (users and tracks) such that edges only connect vertices from different sets (users to tracks, never user-to-user or track-to-track)

```
graph = defaultdict(set)
```

```
for play in plays:
    user = play['user_id']
    track = play['track_id']
    graph[f"user_{user}"].add(f"track_{track}")
    graph[f"track_{track}"].add(f"user_{user}")
```

```
return dict(graph)
```

```
user_track_graph = build_user_track_graph(plays[:100]) # Sample
print(f"User-track graph has {len(user_track_graph)} vertices")
```

```
# Count users vs tracks
```

```
users = sum(1 for v in user_track_graph if v.startswith('user_'))
tracks_in_graph = sum(1 for v in user_track_graph if v.startswith('track_'))
print(f"    {users} users, {tracks_in_graph} tracks")
```

Loading from JSON: adjacency list format

```
In [ ]: def save_graph_to_json(graph: Dict[str, Set[str]], filename: str) -> None:
    """Save graph to JSON file."""
    # Convert sets to lists for JSON serialization
    json_graph = {vertex: list(neighbors) for vertex, neighbors in graph.items()}

    with open(filename, 'w') as f:
        json.dump(json_graph, f, indent=2)

def load_graph_from_json(filename: str) -> Dict[str, Set[str]]:
    """Load graph from JSON file."""
    with open(filename, 'r') as f:
        json_graph = json.load(f)

    # Convert lists back to sets
    return {vertex: set(neighbors) for vertex, neighbors in json_graph.items()}

# Demo: save and load a small graph
sample_graph = {
    "A": {"B", "C"},
    "B": {"A", "C"},
    "C": {"A", "B"},
}

# Save to JSON
save_graph_to_json(sample_graph, "sample_graph.json")
print("Saved graph to sample_graph.json")
```

Load back

```
loaded_graph = load_graph_from_json("sample_graph.json")  
print(f"Loaded graph: {loaded_graph}")
```

Building artist collaboration network

Let's build a real graph from Spotify data!

```
In [ ]: def build_artist_network(tracks: dict) -> Dict[str, Set[str]]:
        """Build artist collaboration network from tracks."""
        graph = defaultdict(set)

        # Method 1: Connect artists who have tracks in same genre
        # Group tracks by genre
        genre_to_artists = defaultdict(set)
        for track in tracks.values():
            artist = track['artist']
            # Simplified: assume genre info or use first word of artist name
            # In real data, you'd have genre tags
            genre_to_artists["all"].add(artist) # Simplified

        # Method 2: Connect artists based on co-occurrence in playlists
        # For now, just create sample connections
        artists = list(set(track['artist'] for track in tracks.values()))

        # Sample: connect each artist to a few others (for demo)
        for i, artist in enumerate(artists[:20]):
            # Connect to next artist (circular)
            next_artist = artists[(i + 1) % min(20, len(artists))]
            graph[artist].add(next_artist)
            graph[next_artist].add(artist)
```

```
return dict(graph)
```

```
artist_network = build_artist_network(tracks)  
print(f"Artist network has {len(artist_network)} artists")
```

```
# Find most connected artist
```

```
if artist_network:
```

```
    degrees = {artist: len(neighbors) for artist, neighbors in artist_ne
```

```
    max_artist = max(degrees.items(), key=lambda x: x[1])
```

```
    print(f"\nMost connected: {max_artist[0]} ({max_artist[1]} connectio
```

```
    print(f"Connected to: {list(artist_network[max_artist[0]])[:5]}")
```

Graph statistics

Density: Measures how many edges exist compared to the maximum possible.

- Formula: $\text{density} = 2E / (V \times (V-1))$ for undirected graphs
- Range: 0 (no edges) to 1 (complete graph)
- Low density (<0.1): sparse graph
- High density (>0.5): dense graph

```
In [ ]: def compute_graph_stats(graph: Dict[str, Set[str]]) -> dict:
        """Compute basic graph statistics."""
        num_vertices = len(graph)
        num_edges = sum(len(neighbors) for neighbors in graph.values()) // 2
        degrees = [len(neighbors) for neighbors in graph.values()]

        return {
            'vertices': num_vertices,
            'edges': num_edges,
            'avg_degree': sum(degrees) / len(degrees) if degrees else 0,
            'max_degree': max(degrees) if degrees else 0,
            'min_degree': min(degrees) if degrees else 0,
            'density': (2 * num_edges) / (num_vertices * (num_vertices - 1))
        }

stats = compute_graph_stats(artist_network)
print("\nGraph statistics:")
```

```
for key, value in stats.items():
    if isinstance(value, float):
        print(f" {key}: {value:.2f}")
    else:
        print(f" {key}: {value}")

# Density interpretation
print(f"\nDensity {stats['density']:.4f} means "
      f"{stats['density'] * 100:.2f}% of possible edges exist")
```

Exercise 2: Load and analyze graph from data

Task: Build a "track recommendation" graph based on play patterns.

Requirements:

1. Two tracks are similar if they were played by the same user
2. Build adjacency list (track_id → set of similar track_ids)
3. Compute graph statistics (vertices, edges, density)
4. Find the 3 tracks with the most similar tracks
5. Print track names and similarity counts

Hint: Group plays by user, then create edges between all tracks a user played.

Commit required: Commit your solution before class ends.

```
In [ ]: # YOUR CODE HERE
# Build track recommendation graph from play patterns

def build_recommendation_graph(plays, tracks):
    """Build graph where tracks played by same user are connected."""
    # TODO: Group plays by user
    # TODO: For each user, create edges between all their tracks
    # TODO: Return adjacency list
    pass
```

```
# Build graph and find most similar tracks  
# rec_graph = build_recommendation_graph(plays, tracks)  
# stats = compute_graph_stats(rec_graph)
```

Complexity summary

Representation space complexity:

- Edge list: $O(E)$
- Adjacency list: $O(V + E)$
- Adjacency matrix: $O(V^2)$

Operation time complexity:

Operation	Edge List	Adj List	Adj Set	Adj Matrix
Add vertex	$O(1)$	$O(1)$	$O(1)$	$O(V^2)$
Add edge	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Check edge	$O(E)$	$O(\text{degree})$	$O(1)$	$O(1)$
Get neighbors	$O(E)$	$O(1)$	$O(1)$	$O(V)$
Remove vertex	$O(E)$	$O(V + E)$	$O(V + E)$	$O(V^2)$

For sparse graphs (most graphs): Adjacency list wins!

Wrap-up: Graph modeling and ingestion

You've learned:

-  Graphs model relationships between entities (vertices and edges)
-  Edge list: simple but slow for queries
-  Adjacency list: best for most graphs (sparse)
-  Adjacency matrix: only for dense graphs
-  Loading graphs from CSV and JSON files
-  Building real-world graphs from Spotify data
-  Graph statistics: degree, density, connectivity

Next lecture: BFS/DFS for routing and reachability

Don't forget: Commit both exercises before you leave!

Complexity checkpoints

Question 1: You have a graph with 10,000 vertices and 50,000 edges. What's the best representation?

- A) Edge list — smallest space
- B) Adjacency list — fast neighbor lookup
- C) Adjacency matrix — fast edge checks
- D) All equally good

Question 2: What's the space complexity of adjacency matrix for V vertices?

- A) $O(V)$
- B) $O(E)$
- C) $O(V + E)$
- D) $O(V^2)$

Question 3: Why use sets instead of lists in adjacency list?

- A) Sets use less memory
- B) Sets give $O(1)$ edge existence checks
- C) Sets are always faster
- D) Sets maintain order

Answers: B (adjacency list best for sparse), D ($O(V^2)$), B ($O(1)$ membership)

